UNIVERSITÀ DEGLI STUDI DI NAPOLI PARTHENOPE

DIPARTIMENTO DI SCIENZE E TECNOLOGIE
CORSO DI LAUREA IN INFORMATICA

PROGETTO DI PROGRAMMAZIONE III

SYNERGY

CANDIDATI

EDDY PASQUALE CHIACCHIO 0124003211
PASQUALE FALCO 0124003132

Indice

1. Descrizione Synergy..... 2
3. Singleton..... 3
4. Factory Method.....4
5. Composite.....5
6. Strategy.....6
7. Observer..... 7

1. Descrizione Synergy

Synergy è una piattaforma digitale integrata progettata per ottimizzare la gestione dei progetti e la collaborazione all'interno di team. Il sistema permette di superare la dispersione delle informazioni offrendo uno strumento unico per pianificare e monitorare il ciclo di vita di ogni iniziativa.

Gli utenti, previa autenticazione sicura, accedono a una **Dashboard** intuitiva da cui possono gestire i **Progetti**, intesi come contenitori logici di attività, documenti e comunicazioni. I **Project Manager** possono definire singole **Attività** (Task), impostando scadenze e priorità, mentre i **Team Member** aggiornano lo stato dei propri compiti. La collaborazione è garantita da un sistema di notifiche in tempo reale.

2. Scopi del Progetto

L'obiettivo primario di Synergy è migliorare l'efficienza dei flussi di lavoro attraverso:

- **Centralizzazione:** Fornire un unico punto di controllo per tutte le risorse di progetto (task e documenti), eliminando la frammentazione informativa.
- **Tracciabilità:** Monitorare costantemente il progresso delle attività e il rispetto delle scadenze impostate.
- **Controllo degli accessi:** Garantire la sicurezza dei dati tramite un sistema di permessi differenziati basato sui ruoli Admin, Project Manager e Team Member.

3. Singleton

I pattern creazionali nascondono i costruttori delle classi fornendo un'interfaccia per lo stanziamento. Il **Singleton**, in particolare, garantisce che una classe abbia una sola istanza e fornisce un punto di accesso globale.

Struttura e Partecipanti

Nel nostro diagramma, il Singleton è rappresentato dalle classi **DataManager** e **SessionManager**.

Implementazione

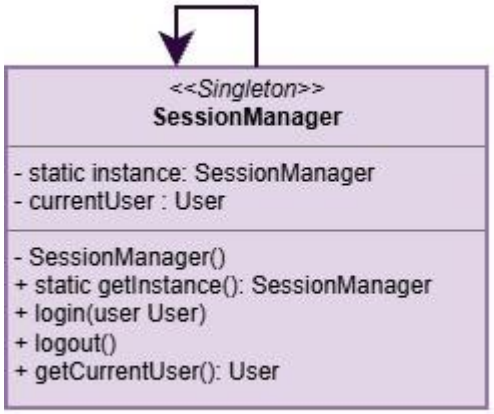
Entrambe presentano un costruttore privato:

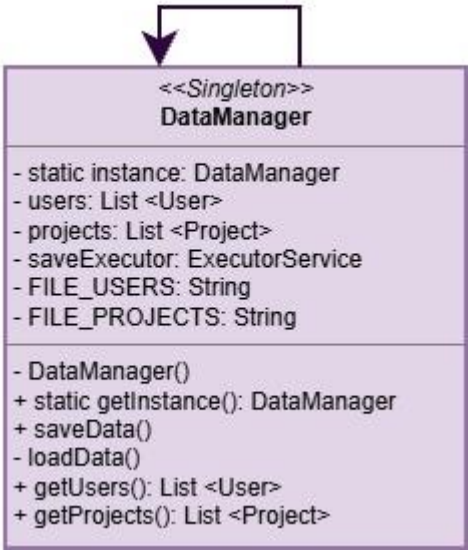
- - DataManager()
- - SessionManager()

e un attributo statico privato (instance) che ne detiene l'unica istanza.

Funzionamento

Il SessionManager assicura che esista un'unica sessione utente attiva per gestire correttamente il login e il logout. Il DataManager centralizza l'accesso alla persistenza dei dati (utenti e progetti), garantendo che tutte le operazioni di salvataggio e caricamento avvengano su un'unica istanza condivisa, evitando conflitti di sincronizzazione.





4. Factory Method

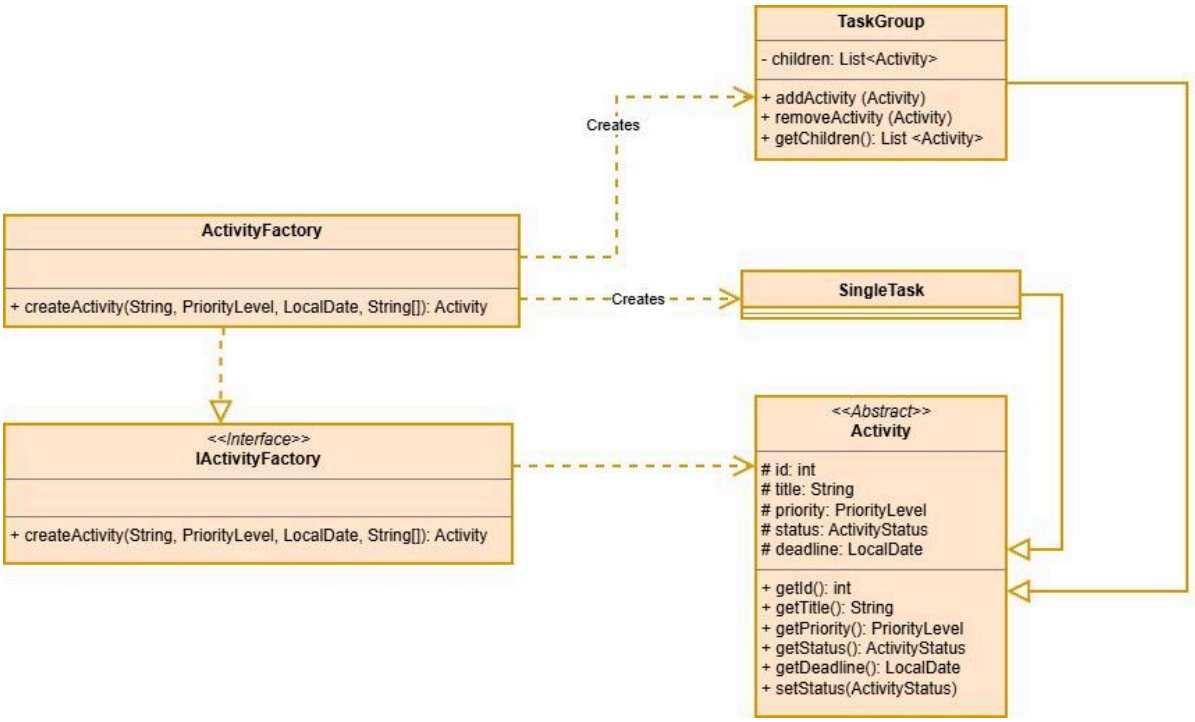
Il **Factory Method** è un pattern creazionale e questo pattern definisce un'interfaccia per la creazione di oggetti, ma delega alle sottoclassi la scelta della classe concreta da istanziare. Serve a disaccoppiare il sistema dai dettagli costruttivi dei prodotti.

Struttura e Partecipanti:

- Factory (Interfaccia): IActivityFactory, che definisce il metodo di creazione.
- ConcreteFactory: ActivityFactory, che implementa la logica di istanziamento.
- ConcreteProduct: SingleTask e TaskGroup.
- Product: La classe Activity che contiene i metodi implementati da ConcreteProduct.

Funzionamento:

Il ProjectController non istanzia mai direttamente un task, ma interroga la factory. Ciò permette di aggiungere nuovi tipi di attività in futuro senza modificare la logica di gestione esistente.



5. Composite

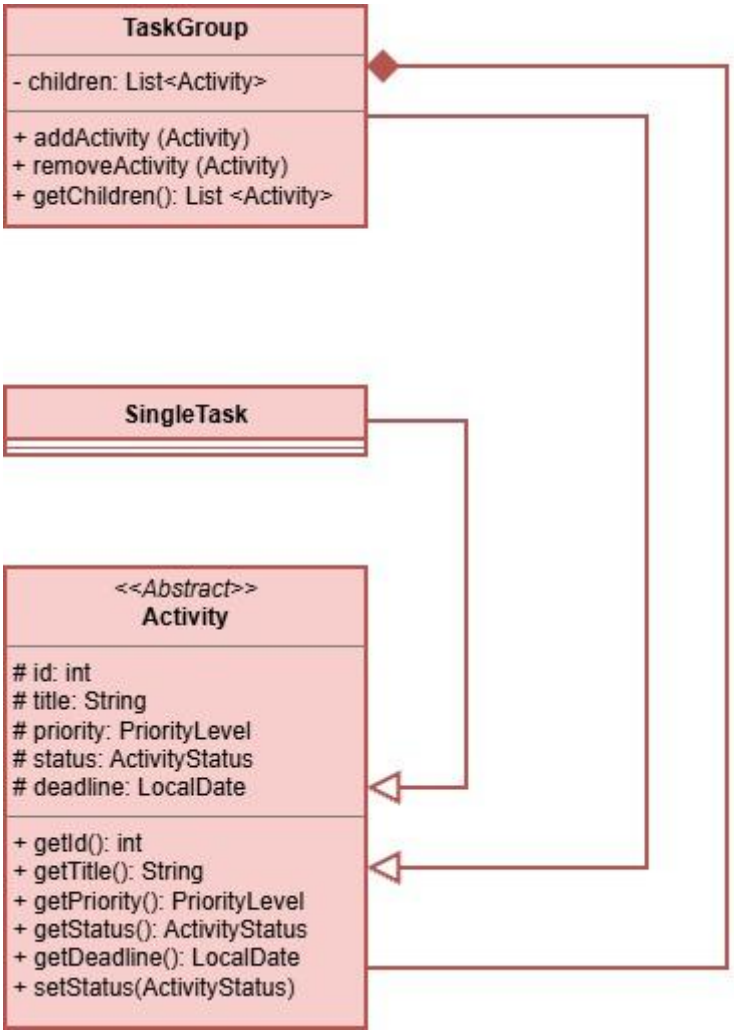
I pattern strutturali si occupano della composizione di classi e oggetti. Il **Composite** permette di trattare oggetti singoli e composizioni di oggetti in modo uniforme, creando gerarchie "parte-tutto".

Struttura e Partecipanti:

- Component: La classe astratta Activity, che definisce l'interfaccia comune.
- Leaf: SingleTask, l'elemento di base che non può avere figli.
- Composite: TaskGroup, che contiene una lista di oggetti Activity.

Funzionamento:

Questa struttura ricorsiva permette a Synergy di gestire progetti con un numero illimitato di sotto-livelli, dove ogni gruppo di task può essere trattato dal sistema come se fosse una singola attività.



6. Strategy

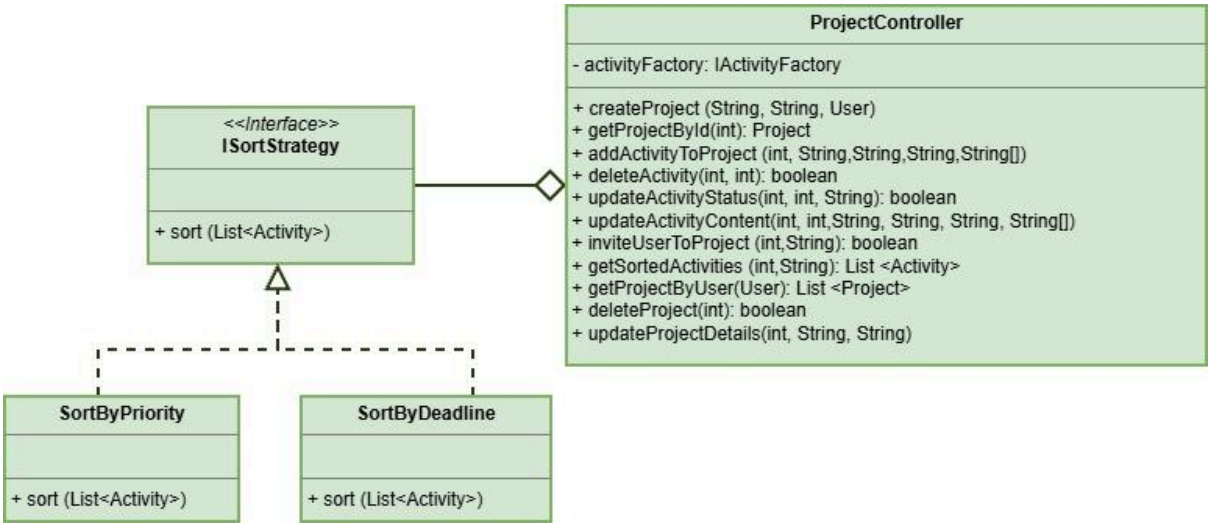
Il pattern **Strategy** definisce una famiglia di algoritmi, li incapsula e li rende intercambiabili, permettendo al client di variare l'algoritmo indipendentemente da chi lo utilizza.

Struttura e Partecipanti:

- Strategy (Interfaccia): ISortStrategy.
- ConcreteStrategy: SortByPriority e SortByDeadline.
- Context: ProjectController, che mantiene un riferimento a un oggetto strategia.

Funzionamento:

È utilizzato nel ProjectController per permettere all'utente di ordinare la lista delle attività di un progetto secondo diversi criteri. Grazie a questo pattern, è possibile cambiare la logica di ordinamento a runtime (per priorità o per scadenza) in modo modulare.



7. Observer

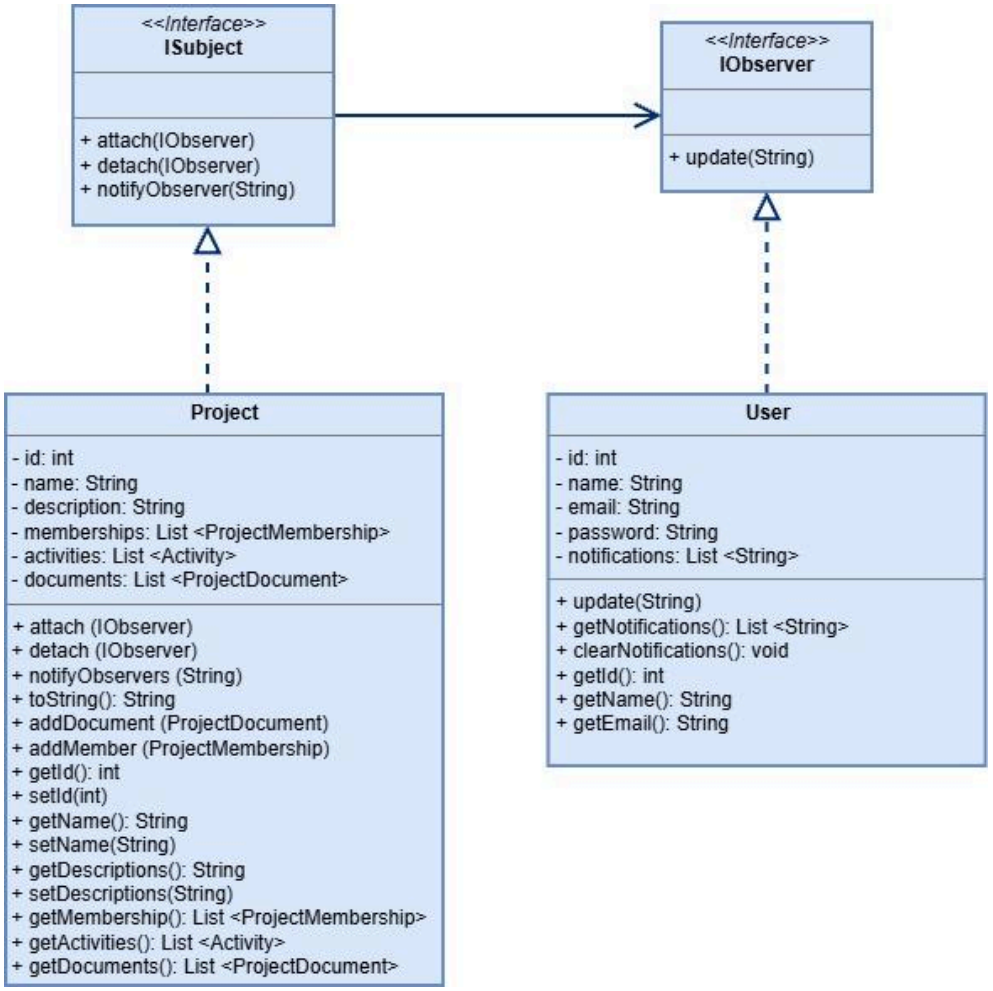
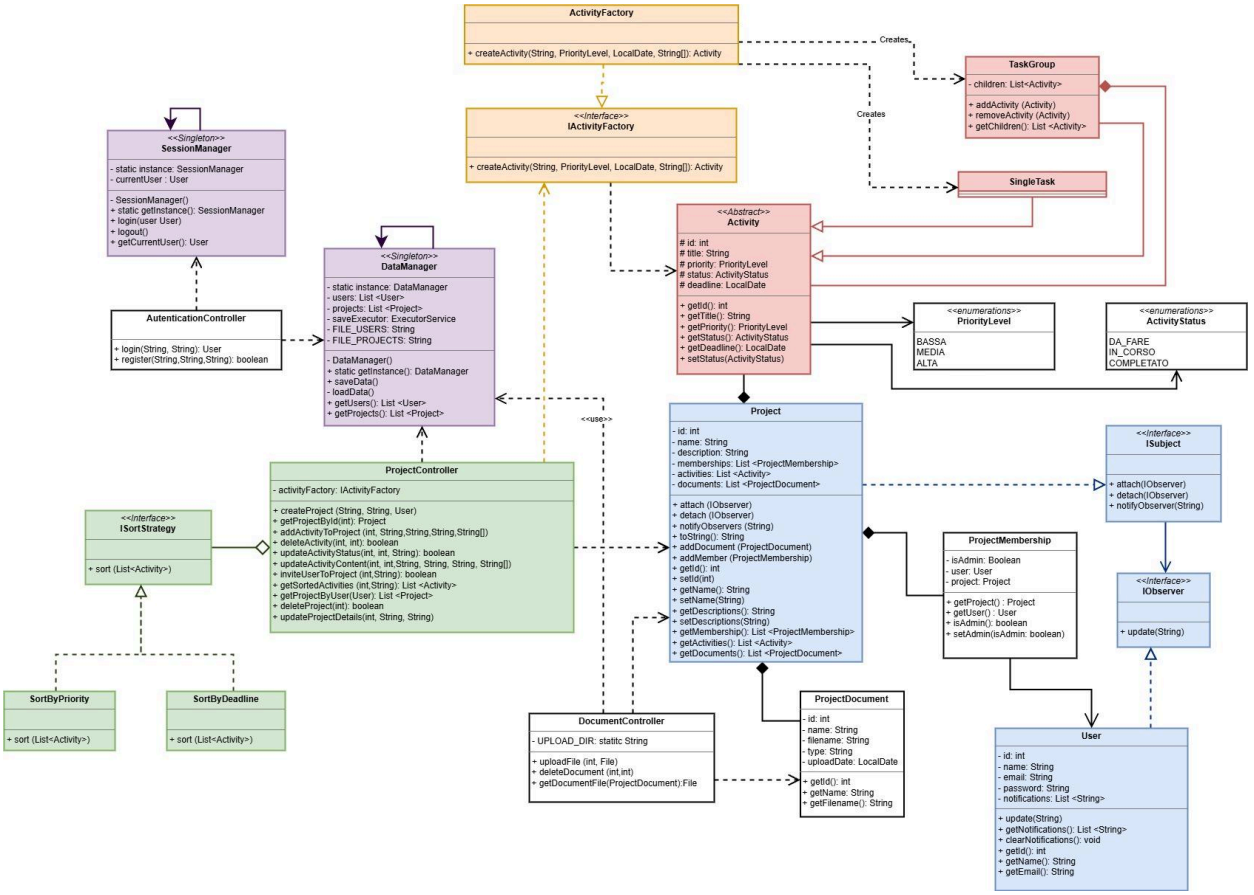
Il pattern **Observer** definisce una dipendenza uno-a-molti tra oggetti, in modo che quando un oggetto (Subject) cambia stato, tutti i suoi dipendenti (Observer) vengano notificati e aggiornati automaticamente.

Struttura e Partecipanti:

- ConcreteObservable: La classe Project che implementa ISubject.
- Observable: La classe ISubject che definisce i metodi per le ConcreteObservable.
- ConcreteObserver: La classe User che implementa IObserver
- Observer: La classe IObserver che definisce i metodi per le ConcreteObserver

Funzionamento:

Quando un progetto viene aggiornato (aggiunta di task o documenti), il metodo notifyObservers() di Project cicla sulla lista degli utenti iscritti, chiamando il loro metodo update(). Questo permette la ricezione immediata delle notifiche nella UI dell'utente senza dover interrogare continuamente il database.



8. Diagramma UML